The Signature Trap: Structural Gaps in Spyware Detection and the Limits of Indicator-Based Defense

Abstract

This paper examines a structural limitation in signature-based spyware detection. The dominant model compares observable artifacts against a curated list of known indicators of compromise. A match proves that a known thing was present. The absence of a match is compatible with two very different situations: nothing is wrong, or something is wrong that nobody has yet described. The delay between an attack's first use and its first published indicator is exactly the period in which signature-based detection reports nothing on an infected device. This paper argues that the correct unit of analysis is not the signature, and not the vulnerability, but the trust assumption the system was built on. When the assumption fails, the entire class of attacks that exploit it is enabled. The paper examines fourteen attack vectors in order of sophistication, from zero-click exploit chains through baseband signalling, kernel persistence, cloud compromise, living-off-the-land abuse, supply chain, physical access, the boot chain, direct memory access, firmware implants, SIM and radio-layer interception, machine-learning model integrity, and physical side channels, identifying for each the trust assumption that fails, why signature-based detection cannot close the resulting gap, and the shape of the defense that would. It expands a four-layer framework for what remains observable from a device the user controls, states why these gaps fall hardest on large populations under weak legal oversight, and states plainly what device-side detection cannot do. The paper does not propose a solution to the hardest problems; it identifies them, names the defense each would require even where that defense does not yet exist, and states its own limits. Its credibility depends on this honesty.

Reading Guide

This paper serves three audiences, and each should enter it at a different point.

A researcher should begin with Section 3, which states the paper's core claim: that the correct unit of analysis in spyware detection is the trust assumption a system was built on, not the signature and not the vulnerability. Section 6, the four-layer framework, and Section 2, the vector analysis, follow from that claim. The contribution a researcher is asked to weigh is the framing, not the enumeration.

A policymaker should begin with Section 7, which states plainly what device-side detection cannot do, and then read Section 8 on the Global South and the regulation subsection of Section 9. The point for a policymaker is narrow and load-bearing: an obligation anchored at the device cannot reach conduct that occurs in the carrier network, the cloud, or the kernel, because the device never observes that conduct. A mandate written as if it can will produce paper compliance and no coverage.

A procurement officer should begin with the procurement subsection of Section 9 and then read Section 7. The point for procurement is a single discipline: a product that reports "no findings" without stating what it examined is reporting absence, not evidence. The right question to a vendor is not "did it find anything" but "what did it look at, and what could it not look at."

The paper is not a sales document. It is a research paper that happens to support a tool. The tool is downstream of the paper. The argument stands on its own.

1. Introduction: The Signature Trap

The cybersecurity industry operates predominantly on a signature-based model. Known indicators of compromise, domain names, file hashes, process names, network addresses, are published, distributed, and matched against observable system state. A match is treated as evidence of compromise. The absence of a match is treated, too often, as evidence of safety. This second inference is not supported by the method. A signature list contains only what has already been observed, analyzed, and published. It cannot contain an indicator for an attack that has not yet been discovered. A scanner that reports “no findings” on a device infected with a novel implant is not malfunctioning. It is producing the only result its method permits.

The published account of FORCEDENTRY, a zero-click iMessage exploit attributed to NSO Group, illustrates the temporal gap. Citizen Lab’s analysis, published in September 2021, concluded that the exploit had been in use since at least February 2021, and that it had been deployed against a Saudi activist before any public indicator existed (Citizen Lab, “FORCEDENTRY: NSO Group iMessage Zero-Click Exploit Captured in the Wild,” September 13, 2021). During those months, no indicator for it existed to be matched. A signature-based scanner running on an affected device during that period would have found nothing. The same structural delay applies to every attack before its first public disclosure. It is not a defect that better engineering removes. It is a property of the method.

This paper argues that the correct unit of analysis is neither the signature nor the vulnerability, but the trust assumption the system was built on. Every attack vector examined here exploits an assumption that was valid at design time and is false at the time of exploitation. SS7’s signalling architecture assumes that all interconnected carriers are honest. That assumption was reasonable when the network was small and national. It is false when a Global Title can be leased by an entity that is not a carrier (GSMA, “FS.52 Global Title Leasing Code of Conduct,” March 2026). iMessage assumes that parsing arbitrary input is a feature requirement. It is, and that is why the attack surface cannot be removed without removing the feature. Signature-based detection assumes that attackers reuse known infrastructure. Sophisticated attackers do not; they rotate infrastructure before signatures are published. When a trust assumption fails, the entire class of attacks that exploits it is enabled. Signatures address the symptom. Structural analysis addresses the cause.

This paper is a companion to an earlier work on signal provenance in retail algorithmic trading, which identified the same structural form in a different domain (Lokesh Gupta, “Signal Provenance in Retail Algorithmic Trading: An Architectural Gap in SEBI’s 2026 Framework and Its General Form,” SSRN, July 2026). There, a regulatory framework anchored accountability at the broker, and the conduct of interest, the origin of a trading signal, occurred upstream of the broker. The framework could not observe what the broker was never sent. It reconstructed signal chains after the fact, from screenshots, bank trails, and messaging records. Here, the anchor is the device’s userspace, and the conduct of interest, the attack itself, occurs in the baseband, the cloud, the kernel, or through legitimate system mechanisms. A device-side tool operates in the same trust domain as the attacker and cannot observe what the attacker has hidden. Both are cases in which accountability is anchored downstream of causation. The

diagnostic is the same: what does the anchored entity actually receive, and is the conduct of concern present in what it receives?

The paper proceeds in three parts. Section 2 examines fourteen attack vectors in order of sophistication, identifying for each the trust assumption that fails and the defense that would close the gap. Section 3 develops the structural argument: the unit of analysis is the trust assumption, not the signature. Section 4 explains why signature-based detection cannot close the resulting gaps and ties the spyware case to the companion paper's reconstruct-versus-observe frame. Sections 5 through 7 describe what remains observable from a device the user controls, expand a four-layer framework for structural detection, and state plainly what device-side detection cannot do. Section 8 explains why these gaps fall hardest on large populations under weak oversight. Section 9 draws the implications for research, regulation, and procurement.

The paper does not propose a solution to the hardest problems. It identifies them, names the shape of each fix, and states its limits. Its credibility depends on this honesty.

2. How Sophisticated Attackers Actually Operate

The following sections examine fourteen attack vectors in order of sophistication. For each, the paper identifies the trust assumption the system was built on, explains why the assumption fails, states why signature-based detection cannot close the gap, and names the defense that would, even where that defense does not exist commercially, requires hardware change, or requires regulatory intervention. Primary sources are cited where available. Where a claim rests on a secondary description rather than a primary document reviewed for this paper, it is marked [unverified] and listed in the appendix. The inventory is deliberately lean; the contribution of this paper is the framework and the limits, not the enumeration of attacks. The structural argument that follows in Section 3 depends on the pattern these vectors share, not on the completeness of the list. The vectors are described at the level of the trust assumption they break, not as operational procedure.

Three classes of attack are deliberately outside this paper's scope. Social engineering and credential phishing target human trust rather than a trust assumption in a system's architecture, and belong to a different framework. Bluetooth and WiFi proximity attacks (BlueBorne, KNOB, and related) are radio-layer vectors distinct from the cellular vectors in Section 2.12, and warrant their own treatment. SIM-swap fraud is a carrier-side account-takeover vector that is adjacent to but distinct from the baseband attacks in Section 2.3. Their exclusion is a scope decision, not a claim that they do not matter.

2.1 Zero-Click Exploit Chains and the Parse Obligation

A zero-click exploit achieves code execution without any action from the victim. The device receives a message, and the message itself is the attack. No link needs to be tapped, no attachment opened, no permission granted. The two most extensively documented instances against iMessage are FORCEDENTRY and BLASTPASS, both attributed to NSO Group and both captured in the wild by Citizen Lab at the University of Toronto.

FORCEDENTRY was discovered while Citizen Lab analyzed the phone of a Saudi activist infected with NSO Group's Pegasus spyware (Citizen Lab, "FORCEDENTRY," September 13, 2021). The exploit targeted Apple's image rendering library and was effective against iOS,

macOS, and watchOS devices. Citizen Lab concluded that the exploit had been in use since at least February 2021, before any public indicator existed. The vulnerability was fixed on September 13, 2021 in iOS 14.8 as CVE-2021-30860 (CERT-EU, “Security Advisory 2021-049,” September 14, 2021). Google Project Zero’s technical analysis described it as “one of the most technically sophisticated exploits we’ve ever seen” (Google Project Zero, “A deep dive into an NSO zero-click iMessage exploit: Remote Code Execution,” December 15, 2021). The mechanism reached a vulnerable decoder through the media pipeline and defeated BlastDoor, the sandbox Apple had introduced to isolate message parsing.

BLASTPASS was discovered in September 2023 while Citizen Lab examined the device of an individual employed by a Washington DC civil society organization (Citizen Lab, “BLASTPASS,” September 7, 2023). The chain compromised iPhones running the latest version of iOS at the time without any interaction from the victim, through attachments sent from an attacker account. Apple issued CVE-2023-41064 and CVE-2023-41061. Citizen Lab reported, and Apple confirmed, that Lockdown Mode blocks this particular attack.

The structural point is not the specific vulnerability in either case. It is that the operating system must parse arbitrary input from untrusted senders. iMessage receives a message and processes its contents to display them. That processing is a feature. A user expects to receive images, documents, and contact cards from people they know and from people they do not. The system cannot decline to parse an incoming message without declining to deliver it. The parse obligation is not a bug that can be patched; it is the feature that makes the messaging application useful. Every parser in the media pipeline, image codecs, PDF renderers, font engines, becomes part of the attack surface because the system is built to accept input from anyone and process it without asking the user first. A system that must interpret untrusted input has an attack surface proportional to the complexity of that interpretation, and that surface cannot be removed without removing the feature.

Why signature-based detection cannot close it. The exploit is novel by construction on first use, so no indicator exists during the window that matters most. The delivery artifact is a legitimate-looking message, and the payload is resident in memory rather than written as a scannable file. By the time a hash or a domain is published, the operator has rotated to the next chain.

What would close this gap. The defense reduces and contains the parse surface rather than removing it. Isolating untrusted parsing in a tightly constrained sandbox with minimal privilege, as BlastDoor attempts, limits what a successful parse exploit reaches. Rewriting media and document parsers in memory-safe languages removes the memory-corruption class that these chains rely on. Hardware memory tagging and pointer authentication raise the cost of turning a parse bug into code execution. A user-selectable reduced-functionality mode, of which Lockdown Mode is the current example, lets a high-risk user trade features for a smaller surface. None of these closes the class, because the feature that requires parsing untrusted input remains; they shrink the surface and contain the blast radius, and that is the honest ceiling of the fix.

2.2 Messaging and App-Specific Chains

WhatsApp CVE-2019-3568 is a second instance of the same structural condition, in a different application. The vulnerability was a buffer overflow in WhatsApp’s VOIP stack that allowed remote code execution via a crafted series of packets sent to a target phone number (National Vulnerability Database, “CVE-2019-3568”; Check Point Research, “The NSO WhatsApp

Vulnerability: This is How It Happened," May 14, 2019). The affected versions spanned WhatsApp for Android, WhatsApp Business for Android, and WhatsApp for iOS among others. The attack required no user interaction; the victim did not need to answer the call. It was attributed to NSO Group and used to install Pegasus (Dark Reading, "WhatsApp Secures Ban on NSO Group After 6-Year Legal Battle," October 23, 2025).

The pattern is the same as iMessage's. WhatsApp's core function is to receive and process communications from anyone who has the user's phone number. The VOIP stack must handle incoming call signaling. That handling is the feature. The attack surface is the feature itself. A messaging application cannot refuse to process incoming packets without refusing to function as a messaging application. The specific vulnerability changes; the structural condition does not.

Why signature-based detection cannot close it. As with iMessage, the exploit is delivered through a legitimate application's normal traffic, executes in memory, and is unique to the operator until it is captured and analyzed. No prior indicator describes it.

What would close this gap. The fix is the same family as Section 2.1: memory-safe reimplementation of the call and media stacks, aggressive privilege reduction and sandboxing of the code that touches unauthenticated network input, and pre-call gating that limits which senders can reach the processing path. An application-level defense that helps here and does not help the general case is stronger authentication before a session's media path is opened, so that unsolicited traffic from an arbitrary number cannot reach the vulnerable stack in the first place.

2.3 Baseband and Signalling Attacks

The previous two vectors involve the operating system parsing input that arrives through a user-facing service. The baseband and signalling layer is different in kind. The cellular modem operates independently of the application processor, runs its own firmware, and communicates with the network using protocols that predate the smartphone by decades. The attack surface here is not a parser that can be sandboxed or a media pipeline that can be hardened. It is the signalling plane itself, and the device has no visibility into it.

SS7, or Signalling System No. 7, is the protocol suite that mobile networks use to exchange information for call routing, SMS delivery, roaming, and subscriber authentication. It was designed in an era when telecommunications networks were operated by a small number of state-owned monopolies, and its security model reflects that origin. The protocol includes no mechanism for authenticating the source of a signalling message and no encryption of signalling traffic between networks (3GPP, "3G TS 33.102 V2.0.0: 3G Security; Security Architecture," April 1999). The trust assumption embedded in the architecture is that every entity connected to the signalling network is a legitimate carrier with no incentive to attack another carrier's subscribers. That assumption was reasonable when the network was small, national, and operated by a handful of incumbents. It is false when access to the signalling plane can be obtained by entities that are not carriers at all.

The mechanism by which that access is obtained is Global Title leasing. A Global Title is an address used in SS7 routing that identifies a signalling point without reference to a specific network node. Operators assign Global Titles to their network elements so that messages can be routed between networks. In a properly controlled environment, only the operator itself would use its own Global Titles. In practice, operators have leased Global Titles to third parties,

sometimes without verifying that the lessee is a legitimate carrier (GSMA, “FS.52 Global Title Leasing Code of Conduct,” March 2026). A lessee with a Global Title can send signalling messages that appear to originate from a legitimate network node. The receiving network, applying the trust assumption that any message from a known Global Title is genuine, processes the message. The GSMA has described Global Title leasing as the single most significant enabler of SS7 attacks (GSMA, “How penetration testers should approach GT Leasing,” January 2025) [unverified].

The consequences include location tracking, call interception, SMS interception, and subscriber profile manipulation. A legitimate query used by a home network to locate a roaming subscriber can be issued by a lessee with a Global Title; the visited network responds with the subscriber’s location, and the subscriber’s device is never notified. The phone does not know it was queried. No artifact is written to the device’s storage. No log entry is generated in the application processor. The attack is invisible to any tool running on the phone because the phone was never a party to the exchange (GSMA, “FS.11 SS7 Interconnect Security Monitoring and Firewall Guidelines,” members only; ITU, “Technical Report on SS7 Vulnerabilities and Mitigation Measures”) [unverified].

Diameter, the protocol used in LTE and 5G networks, inherited SS7’s trust assumptions. It was designed as a more modern replacement, with better handling of authentication and routing, but its interconnect security model still assumes that peer networks are honest. The GSMA’s FS.19 specification defines filtering layers that operators should implement at the interconnect boundary to mitigate Diameter attacks (GSMA, “FS.19 Diameter Interconnect Security,” members only) [unverified]. Those mitigations are placed in the network, not on the device. The phone remains blind. Citizen Lab’s 2026 report, “Bad Connection,” documents covert surveillance campaigns that exploited SS7 and Diameter to track targets’ locations across borders and maps live attack telemetry to operator identifiers and interconnect routes (Citizen Lab, “Bad Connection: Uncovering Global Telecom Exploitation by Covert Surveillance Actors,” Report No. 192, April 23, 2026). It confirms from observed data what the specifications imply: the attack surface is real, it is active, and it is entirely outside the device’s field of view.

Why signature-based detection cannot close it. There is no signature to publish because there is no artifact on the device to match against. The exchange occurs between network elements, at a layer the handset does not see and does not log.

What would close this gap. The defense is entirely carrier-side and regulatory. Signalling firewalls at the interconnect boundary, category-based filtering of the message types that have no legitimate cross-border use, and continuous interconnect monitoring, as the GSMA’s FS.11 and FS.19 guidelines describe, move detection to the only layer that observes the traffic. Governance of Global Title leasing, so that signalling access cannot be resold to non-carriers, closes the enabling condition rather than the symptom. Home-routing of location and SMS queries denies the external querier a direct answer. On newer networks, standalone 5G with proper concealment of the subscriber identity removes some of the identifiers these attacks harvest. Every one of these is an obligation on a carrier or a regulator, not a capability a device can be given, which is precisely the point Section 9 draws for regulation.

2.4 Kernel-Level Persistence

Once an attacker achieves code execution in the kernel, the device's trust model inverts. Every userspace process, including any detection tool running on the device, operates at a privilege level below the kernel. The kernel controls the system call interface, the process table, the file system, and the network stack. A kernel-level implant can intercept any system call and return falsified results. A scanner that asks the kernel for a list of running processes receives whatever the kernel chooses to report. If the implant has hidden a process, the scanner sees nothing. If the implant has altered file system metadata, the scanner sees clean. If the implant has redirected traffic, the scanner sees normal connections.

This is not a hypothetical property. Academic research on kernel-level rootkit detection has consistently shown that detection from within the same privilege domain as the attacker is structurally unreliable. A 2025 paper proposes an eBPF-based mechanism for detecting hidden kernel objects, but its approach depends on the integrity of the eBPF subsystem itself (Yu, Hung, and Chou, "Kernel-level hidden rootkit detection based on eBPF," Computers and Security, vol. 157, October 2025). If the attacker has compromised the kernel before the detector is installed, or can modify the eBPF program afterward, the detection mechanism is subject to the same trust problem. Earlier work on data structure invariants and hypervisor-assisted detection reaches the same conclusion: detection must operate at a privilege level above the attacker to be reliable (Baliga, Ganapathy, and Iftode, "Detecting Kernel-Level Rootkits Using Data Structure Invariants," IEEE Transactions on Dependable and Secure Computing, vol. 8, no. 5, 2011).

Apple's Secure Enclave is a hardware-based key manager isolated from the main processor (Apple, "Protecting keys with the Secure Enclave"). It provides a separate execution environment for cryptographic operations and biometric data and protects keys even if the main operating system is compromised. It is not a general-purpose detection engine; it does not monitor the kernel for malicious modification and does not let a userspace tool verify the running kernel in real time. Android's Verified Boot verifies the integrity of the boot chain from an immutable hardware key up to the system partition (Android Open Source Project, "Compatibility Definition"). But Verified Boot verifies the system image at boot time, not the running kernel at runtime. A kernel-level implant that loads after boot, or that modifies the kernel in memory without altering the on-disk image, is outside its scope.

Why signature-based detection cannot close it. On a compromised device, the detector's view of the system is mediated by the very component that has been compromised. No signature helps, because the kernel decides what the scanner is allowed to see.

What would close this gap. The defense requires a verifier at a privilege level above the attacker. A hypervisor that measures kernel integrity from below, a separate security processor that attests the kernel's state, or hardware-assisted runtime attestation surfaced to the user, each moves the observation point above the kernel. Silicon-enforced protection of kernel memory, of which recent page-protection and secure-monitor designs are examples, narrows what a kernel-level implant can alter. The essential move is architectural: real-time attestation that a device the user holds can present, and that a compromised kernel cannot forge because it is not the component producing the attestation. Absent that, no userspace tool can defend itself against a compromised kernel, and it should not claim to.

2.5 Cloud-Side Compromise

The user's data does not exist only on the device. iCloud Backup stores a copy on Apple's servers. Google account services store contacts, email, calendar, and device backups on Google's servers. WhatsApp cloud backup stores message history on either iCloud or Google Drive. An attacker with the user's account credentials, or with the ability to compel the service provider through legal process, can read that data without touching the device.

Apple's iCloud architecture encrypts data in transit and at rest. Standard Data Protection, the default, encrypts iCloud data and stores the keys in Apple data centers (Apple, "iCloud Data Security Overview"). Advanced Data Protection, an optional setting, extends end-to-end encryption to additional data categories including iCloud Backup, so that only trusted devices hold the keys and Apple cannot access the data. But Advanced Data Protection is not the default. A user who has not enabled it relies on the provider's operational security and legal posture. A device-side tool cannot observe what happens to data after it leaves the device. It cannot see an account takeover, a credential compromise, or a lawful access request. The local copy may be intact and unmodified while the cloud copy is read by someone else.

Why signature-based detection cannot close it. No artifact on the device reflects a cloud-side read. The conduct occurs on infrastructure the device does not control and cannot query.

What would close this gap. The defense is account-side and provider-side. End-to-end encryption enabled by default, rather than as an option a targeted user must know to turn on, removes the provider's ability to disclose readable content. Access logging and login alerting surfaced to the user let account activity be observed at the layer where it happens. Hardware-key-bound sessions make stolen credentials insufficient on their own. Transparency reporting and, where law permits, notice to the affected user, address the compelled-disclosure path. None of this is observable or enforceable from the handset; the obligation attaches to the account and the service provider.

2.6 Living-Off-the-Land

Mobile Device Management profiles are the canonical example of living-off-the-land. An MDM profile is a configuration file that an organization installs to manage settings, enforce policies, and install applications. Apple describes configuration profiles as XML files processed by the operating system (Apple, "Profile," Apple Developer Documentation). An actor who obtains the ability to install a profile can use it to exfiltrate data, install applications, or monitor activity. The profile itself is a legitimate artifact. It is signed, installed through a system-provided mechanism, and appears in settings as a legitimate configuration. No signature can distinguish legitimate enrollment by an employer from abusive use, because the distinction is in the intent of the installer, not in the artifact.

Accessibility APIs present the same problem. They are designed to let assistive technologies read screen content and simulate input, and they are essential for users with disabilities. They are also a powerful surveillance capability when abused. An application with accessibility permission can read everything on the screen, including messages, passwords, and financial information. The grant is visible and the use is logged, but the log records that the application used accessibility services, not why. The mechanism is the feature, and no signature can tell abuse from intended use.

Why signature-based detection cannot close it. The malicious and the legitimate use produce identical artifacts. There is nothing to match on, because the difference lives in intent, which no on-device artifact records.

What would close this gap. The defense is provenance and policy at the point of grant. Making the installer of a profile, and the time and channel of installation, visible and attested to the user turns an invisible grant into an accountable one. Default-deny for the highest-power permissions, with periodic re-consent rather than a single permanent grant, limits the standing capability an abuser can acquire silently. Enterprise enrollment that is cryptographically attested to a known organization separates real MDM from an attacker's imitation of it. The fix is not detection of a bad artifact; it is a system that records and surfaces who asked for the power and when.

2.7 Supply Chain

Supply chain attacks exploit indistinguishability at the code level. A malicious library compiled into a legitimate application looks like the application. The application is signed by the developer, distributed through the official store, and passes automated review. The malicious component is compiled into the binary, so it is not a separate file that can be scanned. Public incidents involving malicious mobile SDKs and compromised package ecosystems have shown that hostile code can enter widely used dependencies and reach the applications built on them (Safeguard, "Malicious iOS SDKs and CocoaPods: A Security Report," November 10, 2025; NowSecure, "axios Was Compromised. Your Mobile App May Have Been Too," April 2, 2026) [unverified]. Signature-based detection cannot see the difference between the application as published and the application with the malicious component compiled in, because the malicious component is not a separate signature-bearing artifact. It is part of the signed binary.

Why signature-based detection cannot close it. The signature over the binary is valid; the developer really did sign it. The hostile code is inside the thing the signature vouches for, so the signature attests to the wrong property.

What would close this gap. The defense is integrity and provenance at the distribution layer, not on the device. Reproducible builds let a third party confirm that a published binary corresponds to reviewed source. A software bill of materials makes the set of included dependencies explicit and auditable. Signed build provenance, of the kind supply-chain integrity frameworks describe, binds an artifact to the pipeline that produced it. Binary transparency logs make after-the-fact substitution detectable. Store-side re-scanning of compiled binaries, rather than one-time review, shortens the window in which a poisoned dependency ships unnoticed. The device cannot perform any of this, because by the time the binary reaches the device the compromise is already inside a validly signed package.

2.8 Physical Access

Physical access is the simplest vector and the most difficult to defend against. A short period with an unlocked phone is sufficient to change its configuration. The actor does not need an exploit chain. They need possession of an unlocked device and the ability to install an application, configure a profile, or enable a management mode. Lockdown Mode reduces this surface. Apple states that when Lockdown Mode is enabled, configuration profiles cannot be installed and the device cannot be enrolled in Mobile Device Management or supervision (Apple, "About Lockdown Mode"). It also blocks most message attachment types, disables certain web

technologies, and turns off 2G and 3G support. It does not eliminate the surface. It reduces it, and it does not protect against an actor who has the user's passcode.

Why signature-based detection cannot close it. A configuration change made through legitimate mechanisms leaves legitimate artifacts, and a change made with the passcode is indistinguishable from a change the owner made. After-the-fact, the device cannot reliably tell that its own recent state was set by someone else.

What would close this gap. The defense narrows what possession alone can accomplish. Hardware-enforced restriction of data ports while the device is locked, of which USB-restricted modes are the current example, denies the fast physical path. Strong passcodes and biometric gating raise the bar for the possession-plus-passcode case. Attestation that would reveal a profile installation or a management enrollment to the legitimate owner turns a silent change into a visible one. Tamper-evident hardware and duress or ephemeral modes address the coercion case. The honest limit remains: an actor with possession and the passcode has the owner's own authority, and no device-side tool can fully distinguish that from the owner acting.

2.9 Boot Chain and Secure Boot

The trust assumption is that the operating system the device boots into is the operating system the manufacturer shipped. Secure Boot exists to enforce exactly this: each stage of startup verifies the signature of the next before handing control to it, anchored in a key the manufacturer controls. When the chain holds, everything above it, including every detection tool, runs on a known-good base.

The assumption fails when a link in the chain is signed but vulnerable, when the key hierarchy can be abused, or when the verification can be disabled below the level the operating system can see. Publicly documented boot-level compromises have shown that a signed-but-flawed bootloader component can be used to run unverified code before the operating system starts, and that revoking the vulnerable component is slow and risky because a botched revocation can render devices unbootable (for example, the BootHole GRUB2 bootloader flaw reported by Eclypsium in 2020, CVE-2020-10713, and the BlackLotus UEFI bootkit that bypasses Secure Boot on fully patched systems, reported by ESET in 2023, CVE-2022-21894). A bootkit that establishes itself below the operating system controls the environment the operating system later runs in. It can present a system that looks entirely stock to anything running above it.

Why signature-based detection cannot close it. The compromise is pre-operating-system. Any scanner runs after the boot chain has already handed over control, and it inspects a system whose reported state is chosen by the layer beneath it. There is no on-device artifact that a compromised boot chain must expose, because the compromised chain decides what is exposed.

What would close this gap. The defense is measured boot with an external verifier and robust revocation. A hardware root of trust that measures each boot stage into a protected register, and reports those measurements to a verifier the compromised chain cannot forge, lets the user or an enterprise learn that the booted system does not match the shipped one. Anchoring the first verification in immutable ROM rather than in updatable firmware removes the lowest rung as a target. A revocation mechanism that can retire a vulnerable signed component without bricking honest devices is what makes the model maintainable in practice. Attestation of the boot

measurements, surfaced to the user, is the piece that turns Secure Boot from a local check the attacker can subvert into evidence the user can inspect.

2.10 DMA and Peripheral Attacks

The trust assumption is that attached hardware is what it claims to be, and that a peripheral granted direct access to memory will use it only as intended. Direct Memory Access exists for performance: it lets a device move data to and from system memory without routing every transfer through the processor. A high-speed port that offers DMA to whatever is plugged into it extends a great deal of trust to the thing on the other end of the cable.

The assumption fails when the attached device is hostile. Public research on Thunderbolt and other DMA-capable interfaces has shown that a malicious peripheral can read and write system memory in ways the operating system does not mediate, and can do so in configurations where the protective hardware unit is absent, disabled, or incompletely applied (for example, the Thunderclap research presented at NDSS in 2019, and the Thunderspy research published by Ruytenberg in 2020). Because the transfer is hardware-to-memory, it can bypass software access controls that assume all memory access flows through the processor.

Why signature-based detection cannot close it. A DMA read leaves no software artifact to scan, and the malicious actor is a physical device rather than a file. Nothing in the file system or the process table records that memory was read over the bus. The attack lives below the layer any on-device scanner observes.

What would close this gap. The defense is memory isolation enforced in hardware and authentication of peripherals. An input-output memory management unit that confines each DMA-capable device to a narrow, pre-approved region, enabled by default on every DMA-capable port rather than left optional, removes the arbitrary-memory access that these attacks depend on. Gating DMA while the device is locked, and requiring explicit user approval before a newly attached high-speed peripheral is trusted, closes the drive-by case. Device attestation, in which a peripheral proves what it is before it is granted access, addresses the impersonation at the root. This is a hardware and platform-design fix; a scanner cannot supply it.

2.11 Firmware and Component Implants

The trust assumption is that firmware is immutable, or at least trustworthy: that the low-level code running the main board, the storage controller, the radio, and the many small components in a modern device is the code the manufacturer intended, and stays that way. The operating system sits on top of this firmware and trusts it implicitly.

The assumption fails because firmware is writable, sits below the operating system’s view, and is rarely verified at runtime. Public research and disclosed incidents have documented firmware-level implants that persist beneath the operating system, including a UEFI rootkit found in the wild that survives operating-system reinstallation (the LoJax implant reported by ESET in 2018), and analyses describing malicious code hidden in the firmware of storage-device controllers, both as an academic proof of concept that compromises a commodity drive’s firmware using public information and reverse engineering (Zaddach and colleagues, ACSAC 2013) and as an in-the-wild capability that reprograms drive firmware across many vendors and survives an operating-system reinstall (documented in Kaspersky’s analysis of the Equation Group, 2015). A

hostile replacement component, a screen, a battery, or a charging controller carrying its own microcontroller, extends the same problem to the physical supply of parts: the component the device trusts as passive hardware may carry logic of its own.

Why signature-based detection cannot close it. Firmware lives below the file system. An operating-system scan cannot read the running firmware of a peripheral controller, and a firmware implant can present clean state to anything that asks from above. There is no file to hash and no process to enumerate, because the implant is not in the operating system's world.

What would close this gap. The defense is signed, measured, and attestable firmware, extended down to components. Firmware that is cryptographically signed, measured into the same root of trust that measures the boot chain, and reported through attestation, lets a verifier learn that a component is running code it should not. Authenticating components so that a device can tell a genuine part from a hostile replacement addresses the supply-of-parts case. Provenance for components, tracked from manufacture, is the supply-chain analogue of the software bill of materials in Section 2.7. As with the boot chain, the enabling move is attestation the user can inspect, because the one thing a firmware implant cannot easily do is forge a measurement produced by hardware beneath it.

2.12 SIM, eSIM, and IMSI Catchers

The trust assumption is twofold: that the tower the phone attaches to is a legitimate carrier tower, and that the SIM, which is itself a small computer with its own operating system, runs only benign code. The handset is built to attach to the strongest available cell and to obey certain commands from the SIM, and historically the network did not have to prove its identity to the phone.

The assumption fails on both halves. A cell-site simulator, often called an IMSI catcher, presents itself as a legitimate tower, induces nearby phones to attach, and can harvest identifiers or force a downgrade to older radio protocols with weaker protections; such devices are widely documented in public reporting and research on mobile-network security [unverified]. On the SIM side, attacks that abuse SIM application-toolkit functionality have been reported, in which a command delivered to a vulnerable SIM application causes the SIM to act without the user's knowledge (the Simjacker technique reported by AdaptiveMobile Security, now Enea, in 2019). In both cases the conduct occurs at or below the radio link, on components the handset's operating system does not fully observe.

Why signature-based detection cannot close it. A rogue base station is not a file, and the identifiers it harvests are exchanged in the attach procedure rather than stored on the device. A malicious SIM command executes on the SIM's own processor, outside the operating system's field of view. There is no on-device indicator that a scanner can match against.

What would close this gap. The defense is mutual authentication and the removal of weak fallbacks, plus carrier-side hardening. Requiring the network to authenticate itself to the device, and refusing silent downgrade to legacy protocols that lack that guarantee, denies the rogue-tower its foothold; on newer networks, concealing the long-term subscriber identifier so it is not broadcast in the clear removes the identifier a catcher seeks to harvest. User-visible warning when a device attaches to an unauthenticated or downgraded cell turns an invisible event into an observable one. On the SIM side, disabling or hardening the toolkit functionality that these

attacks abuse, and applying integrity controls to over-the-air provisioning of SIM and eSIM profiles, closes the SIM path. Much of this is a carrier and standards obligation, which again places the fix off the device.

2.13 AI and ML Model Integrity

The trust assumption is that a machine-learning model behaves as it was trained to behave, and, most pointedly for this paper, that a detector built on a model can be trusted to report what it sees. As detection systems adopt learned models to find behavior that fixed signatures miss, the model itself becomes a component whose integrity must be assumed, and that assumption is not free.

The assumption fails in several documented ways. Adversarial examples show that small, deliberate perturbations of an input can cause a model to produce a confident wrong answer (the phenomenon described by Szegedy and colleagues in 2013 and by Goodfellow, Shlens, and Szegedy in 2015). Data poisoning shows that an attacker who influences the training data can shape the model's later behavior. Backdoor attacks show that a model can be trained to behave normally except on a chosen trigger, at which point it misclassifies by design (the BadNets work by Gu and colleagues in 2017). National-standards work has begun to catalogue these adversarial-machine-learning modes as a distinct class (NIST's taxonomy of adversarial machine learning, NIST AI 100-2e2025). Applied to a detector, each of these means the same thing: the model can be made to say clean when it should say compromised, and it will say so with the same confidence it shows on honest inputs.

Why signature-based detection cannot close it. There is no indicator of compromise for "the classifier was fooled" or "the model was poisoned." The failure is in the decision procedure, not in a file or a domain. A model that has been evaded or backdoored emits a normal-looking output, and a signature over inputs cannot capture a corruption of the mapping from input to output.

What would close this gap. The defense is integrity, provenance, and epistemic humility about the model. Attesting the integrity and provenance of model weights, so that the deployed model is verifiably the trained one, addresses substitution and tampering. Governance of training data addresses poisoning at its source. Adversarial-robustness testing, out-of-distribution detection, and ensembles that must agree, raise the cost of a single evasive input. For a detector specifically, the discipline that matters most is the one this paper argues for throughout: the model's output is evidence carrying stated uncertainty, never ground truth, and a high-stakes negative is escalated rather than trusted. A detector that is itself a learned system inherits this entire gap, and a paper that recommends learned detection is obliged to say so.

2.14 Side-Channel Attacks

The trust assumption is that the device emits signals only when the user intends it to, and only through the channels the user is aware of. In fact every physical system leaks: it draws power in patterns that track what it computes, it emits electromagnetic radiation, it makes sound, and its shared internal resources take different amounts of time depending on data. None of this leakage is a channel the user chose.

The assumption fails because the leakage is inherent to the physics of computation rather than a flaw that can be patched away. Long-standing research on electromagnetic emanations has shown that information can be recovered from a device's unintended radiation (the emanation-eavesdropping work associated with van Eck in 1985, part of the concern known as TEMPEST). Microarchitectural timing attacks have shown that data can leak across supposed isolation boundaries through the shared behavior of the processor (the Spectre and Meltdown class of attacks disclosed in 2018). Power, acoustic, and sensor-based channels extend the same principle. The common thread is that the device reveals something through a channel the user does not regard as an output at all.

Why signature-based detection cannot close it. Leakage is physics, not a file. Nothing on the device records that its electromagnetic radiation was captured across a room, or that a timing difference was measured. There is no artifact, so there is nothing to sign and nothing to match.

What would close this gap. The defense is design-time and physical. Constant-time implementations remove the data-dependent timing that microarchitectural attacks read. Microarchitectural partitioning and isolation reduce cross-boundary leakage inside the processor. Shielding and noise injection reduce electromagnetic and acoustic emanation. For remote emanation capture, physical distance and a controlled environment are the practical control. This is overwhelmingly a hardware and system-design matter; device-side software detection essentially cannot see a side-channel attack in progress, and should not claim to.

3. The Structural Gaps That Enable Them

The fourteen vectors examined above differ in their technical details and their sophistication. They are the same in their structural form. Each exploits a trust assumption that was valid at design time and is false at the time of exploitation.

The correct unit of analysis, then, is not the signature and not the vulnerability. It is the trust assumption the system was built on. When the assumption fails, the entire class of attacks that exploit it is enabled. A signature for a specific exploit addresses one instance. It does not address the assumption that made the exploit possible. The assumption will be exploited again, in a different way, by a different attacker, using a different technique. The signature will not match.

This is the sense in which the gap is structural. It is not a defect in a particular implementation. It is a property of the architecture. The iMessage parse obligation is a property of iMessage being a messaging application. The SS7 trust model is a property of SS7 being designed for a small, trusted network. The kernel's privilege over userspace is a property of the operating system's design. The cloud's separation from the device is a property of cloud services. The indistinguishability of legitimate and malicious mechanism use is a property of the mechanisms themselves. The supply chain's opacity is a property of compiled binaries. The boot chain's authority is a property of the device having to start somewhere. A peripheral's memory access is a property of the performance bargain that DMA exists to make. Firmware's invisibility is a property of its sitting beneath the operating system. The radio's trust in the tower is a property of a network that was not designed to prove itself to the phone. A model's behavior is a property of what it learned. Physical leakage is a property of computation being physical. In each case the vulnerability is downstream of a design decision, and the design decision encodes a trust assumption that an adversary can now falsify.

The companion paper on signal provenance identified the same structural form in a different domain. There, a regulatory framework anchored accountability at the broker, and the conduct of interest, the origin of a trading signal, occurred upstream of the broker. The framework could not observe what the broker was never sent. It reconstructed signal chains after the fact. Here, the anchor is the device's userspace, and the conduct of interest occurs in the baseband, the cloud, the kernel, the boot chain, the firmware, the radio link, or through legitimate system mechanisms. A device-side tool operates in the same trust domain as the attacker and cannot observe what the attacker has hidden. Both are cases in which accountability is anchored downstream of causation. The diagnostic is the same: what does the anchored entity actually receive, and is the conduct of concern present in what it receives?

4. Why Signature-Based Detection Cannot Close These Gaps

Signature-based detection is the dominant model in cybersecurity. It works by comparing observable artifacts against a curated list of known indicators of compromise. The list is built from forensic analysis of past attacks: domain names that implants contacted, file hashes of malicious components, process names, network addresses. When a match is found, the system reports a detection. When no match is found, the system reports nothing. The absence of a match is treated, too often, as evidence of safety.

The structural limit of this model is that a signature list contains only what has already been observed, analyzed, and published. It cannot contain an indicator for an attack that has not yet been discovered. The delay between an attack's first use and its first published indicator is exactly the period in which signature-based detection reports nothing on an infected device. FORCEDENTRY was in use from at least February 2021 until it was captured and analyzed in September 2021 (Citizen Lab, "FORCEDENTRY," September 2021). During those months, no indicator existed to be matched. BLASTPASS was in use before it was captured in September 2023 (Citizen Lab, "BLASTPASS," September 2023). The same temporal gap applies to every attack before its first public disclosure.

The industry continues to invest in signature-based detection despite this limit, for reasons that are rational within its constraints. Signature-based detection works for the vast majority of commodity malware. It is cheap to deploy at scale. It produces measurable results, detection counts and block rates, that are legible to procurement and compliance. It creates a market for indicator feeds and threat-intelligence subscriptions. The equilibrium is stable, and it is not a conspiracy. It is a set of incentives that reward the production and consumption of signatures. But the equilibrium leaves the most sophisticated attacks unaddressed. The attacks that matter most, the ones used against journalists, activists, and lawyers, are the ones least likely to be detected by signature matching, because the actors who deploy them rotate infrastructure, use zero-click chains, and operate in layers the device cannot see.

The objection that better signatures or AI-based anomaly detection might close the gap does not survive the structural analysis. AI-based anomaly detection on the device operates in the same trust domain as the attacker. If the kernel is compromised, the anomaly detector's observations are mediated by the compromised kernel. If the attack occurs in the baseband, the anomaly detector has no data. If the attack exploits a legitimate mechanism, the anomaly detector sees legitimate activity. And, as Section 2.13 sets out, the anomaly detector is itself a model that can be evaded or poisoned. The structural limits that defeat signature matching also defeat device-

side anomaly detection, because both operate at the same layer and are subject to the same trust assumptions.

This is where the parallel with the companion paper on signal provenance becomes exact. In the SEBI case, the regulatory framework anchored accountability at the broker, and the conduct of interest, the origin of a trading signal, occurred upstream of the broker. The framework reconstructed signal chains after the fact, from screenshots, bank trails, and messaging records, because nothing observed the signal at the time it was received. In the spyware case, the detection framework anchors observation at the device's userspace, and the conduct of interest, the attack itself, occurs in layers the userspace cannot see. The framework reconstructs the attack after the fact, from forensic artifacts, because nothing observed the attack at the time it occurred. Both are instances of the same structural gap: accountability is anchored downstream of causation, and the framework reconstructs rather than observes. The diagnostic is the same in both cases: what does the anchored entity actually receive, and is the conduct of concern present in what it receives?

5. What Is Observable from a Device the User Controls

The preceding sections established what cannot be observed from a device-side tool. This section states what can be observed, and under what conditions. The analysis is grounded in artifacts a user can obtain without special access: an iPhone backup, encrypted or unencrypted, and an Android bug report.

An iPhone backup, created with Finder and optionally encrypted with a password, contains user-area data. Apple states that encrypted backups can include information that unencrypted backups do not: saved passwords, Wi-Fi settings, website history, health data, and call history (Apple, "About encrypted backups on your iPhone, iPad, or iPod touch," article 108353). An unencrypted backup holds no browsing history and no call history, so nothing can be concluded about them from it. A decrypted backup contains a manifest listing files, application data, and databases for messages, calls, contacts, browser history, installed configuration profiles, and a record of which programs used the network. Indicators map onto these artifacts: a domain in history or in a link, a file path in the manifest, a program name in the network-usage record, a hash of a stored file.

An Android bug report, captured from a developer option or with a debug bridge command, contains diagnostic and log output as text (Android Developers, "Capture and read bug reports"). It includes the values of system properties, including the security patch level string (Android Open Source Project, "Android Security Bulletin, December 2025"). Other standard properties record whether a build is debuggable or insecure, and the kernel command line carries the bootloader and enforcement state the kernel was started with. Reading these allows a tool to make integrity observations: whether this is a debug build, whether the enforcement mode is permissive, whether the bootloader is unlocked, whether the report mentions a superuser binary at a usual path.

Both artifacts are the device describing itself. A backup contains what the operating system chose to include. A bug report contains what the operating system chose to print. They are evidence about the device's userspace state as the device reports it. That is useful, and it is also the limit of what they can show. They cannot attest to the state of the baseband, the kernel, the

boot chain, the firmware, or the cloud. They cannot show that a jailbreak is absent, because a jailbreak lives on the system partition and the boot chain, neither of which is in a backup. They cannot show that Lockdown Mode is on, because no artifact recording that setting in a backup is known to the author. They cannot show that an MDM profile is malicious, because the profile is a legitimate artifact. They can show what the device reports about its own userspace state, and they can be analyzed for patterns consistent with known spyware, but they cannot close the gaps identified in Section 3. Critically, and this follows from Section 2.4, if the layer that produces these artifacts has itself been compromised, the artifacts report whatever that layer chooses; a self-report is evidence only to the extent the reporter is honest.

6. A Framework for Structural Detection

The preceding analysis establishes that signature-based detection is structurally insufficient for the most sophisticated attacks. This section expands a four-layer framework for what remains observable from a device the user controls. The framework is conceptual and is described independently of any particular implementation. Where a concrete implementation status is relevant, it is cited from an unpublished companion document and marked as such. The organizing principle across all four layers is that no layer may claim more than its evidence supports, and that the absence of a finding is never evidence of safety.

The four layers address the subset of vectors that are observable from a device the user controls. The remaining vectors, baseband, kernel, boot chain, DMA, firmware, SIM, AI and ML integrity, and side channel, require defenses at other layers, as stated in each vector's "What would close this gap" subsection. The framework does not claim to cover them, and the paper does not propose that it can.

Layer 1: Indicator Matching

What it observes. Layer 1 compares observable artifacts against a curated indicator database: domains, file paths, program names, and hashes present in a backup or a bug report. On a hit it produces a high-confidence, precise result, and it names what matched and where. It is the current state of the art for the narrow question of whether any known thing is present, and it is fast and cheap to run.

What it cannot observe. It is blind to novel attacks by construction, because an indicator for an attack that has not been analyzed cannot exist. It sees only the artifacts the device chose to include, so it inherits every limit in Section 5: nothing in the baseband, the kernel, the boot chain, the firmware, or the cloud. Its false-positive character is favorable, because an exact match on a structured indicator such as a hash or a full domain is rarely coincidental, but its false-negative character is the opposite and is the whole problem: a clean Layer 1 result is compatible with a fully compromised device.

Deployment constraints and honesty. A Layer 1 result must state what it examined and what it did not, so that a reader can see that a backup with no browsing history could not have produced a website match. It must refuse rather than return empty when its source is missing, because an empty report reads as "nothing found," which is a claim the missing source cannot support. The companion whitepaper describes an implementation of Layer 1 for iPhone backups, with a prototype validated on synthetic data only, which has never produced a finding on a real device

and for which no detection rate has been measured (whitepaper, Sections 5 and 7). The framework does not depend on that implementation; it is cited so that the claim is traceable.

Layer 2: Permission and Access Observation

What it observes. Layer 2 records which applications hold and use sensitive permissions, when, and under what conditions. It is aimed at the living-off-the-land class in Section 2.6: stalkerware and applications that abuse accessibility services, location, microphone, or camera permissions. Because the abuse and the legitimate use share the same artifact, the observable that matters here is the pattern of use over time, not the mere presence of a grant.

What it cannot observe. It does not detect kernel-level attacks, because the kernel mediates the permission system and can falsify its records. It does not detect baseband, boot chain, or firmware attacks, none of which touch the application permission system. It cannot read intent, so it cannot by itself distinguish an employer's legitimate management from an abuser's, which is exactly the indistinguishability Section 2.6 describes.

False positives, constraints, and honesty. The central risk is a high false-positive rate: many legitimate applications use powerful permissions for good reasons, and flagging them all teaches the user to ignore the tool. Layer 2 therefore has to characterize use rather than presence, which requires time on the device and careful thresholds, and even then it reports a pattern worth a human's attention, not a verdict. It must not describe an application as malicious on the strength of a permission it legitimately holds. Layer 2 is a research direction, not a deployed capability, and it must say so.

Layer 3: Network Metadata Analysis

What it observes. Layer 3 examines the destinations and volumes of network traffic without examining content. It targets a property that most implants cannot avoid: exfiltration and command-and-control require the device to communicate. Regular, periodic connections to a destination, the beaconing pattern, are visible in metadata even when the content is encrypted, and a sudden volume of outbound data to an unfamiliar destination is visible without reading a byte of it.

What it cannot observe. It does not see encrypted content, so it cannot distinguish a legitimate connection to a common cloud service from a malicious connection routed through the same service. An implant that exfiltrates slowly, blends with normal traffic, or tunnels through a widely trusted destination can stay under the threshold. And, as with every device-side layer, if the kernel has been compromised the traffic record itself can be falsified before Layer 3 ever sees it.

False positives, constraints, and honesty. Modern applications beacon constantly for legitimate reasons, so the false-positive surface is large and the useful signal is a matter of degree. Collecting network metadata is also privacy-sensitive, which constrains where the analysis may run and requires that it stay on the device. Layer 3 reports a destination and a pattern that merit scrutiny, not proof of exfiltration, and it is a research direction rather than a deployed capability.

Layer 4: Behavioral Baseline

What it observes. Layer 4 builds a per-device, per-user baseline of normal behavior and reports deviations from it. It is the layer most likely to surface novel spyware that trips none of the first three, precisely because it does not depend on prior knowledge of the attack; it depends on the target's own history.

What it cannot observe. It cannot observe anything outside the userspace it baselines, so the whole of Section 7 remains beyond it. It is also, uniquely among the four, exposed to a slow adversary: an attacker who operates within normal parameters, or who is present during the learning period, can poison the baseline so that malicious behavior is learned as normal. Its observations are still mediated by the operating system and inherit the Section 2.4 problem.

False positives, constraints, and honesty. Layer 4 has the highest false-positive rate of the four, because behavior varies legitimately across users, contexts, travel, and time, and every genuine change looks like a deviation. It requires time on the device to learn a baseline at all, and careful thresholding to avoid drowning the user, and even then it is the layer most in danger of overclaiming, because a deviation is an anomaly and not a diagnosis. It is a research direction, not a deployed capability, and it must never present a deviation as a detection.

The four layers are intended to be complementary. Layer 1 catches known threats with high confidence. Layer 2 catches applications that abuse legitimate APIs. Layer 3 catches exfiltration and beaconing. Layer 4 catches novel behavior that matches no known pattern. No layer is sufficient on its own, and none of Layers 2 through 4 has been built or evaluated, so whether they would detect what spyware does is unknown. They are stated here because they shape the design of Layer 1, in particular its refusal to reduce a result to "clean." Each layer is held to the same standard: it must state its limits, and it must not overclaim.

7. Limits: What Device-Side Detection Cannot Do

This section states plainly what device-side detection cannot do. The paper's credibility depends on it.

Baseband and signalling attacks over SS7 and Diameter cannot be detected from a device-side tool. The phone never knows it was queried, and no device-side artifact exists. The defense is carrier-side or network-side, through signalling firewalls, interconnect monitoring, and governance of Global Title leasing, as the GSMA FS.11, FS.19, and FS.52 guidelines describe. The device is blind. These specifications are member-distributed and were not reviewed in full for this paper; they are cited from secondary descriptions and marked [unverified].

Cloud-side compromise cannot be detected from a device-side tool. The user's data exists in two places and the device sees only one. An actor with account access, or the ability to compel the provider, can read the cloud copy without touching the device. The defense is account-side and provider-side: default end-to-end encryption, access logging, login alerting, and transparency about disclosure.

Kernel-level attacks, once established, can evade userspace detection. The kernel controls the system call interface and can falsify any result returned to a userspace scanner. The detector operates in the same trust domain as the attacker. The defense requires a privilege level above

the attacker: a hypervisor, a separate security processor, or hardware-assisted attestation surfaced to the user.

Boot chain and Secure Boot compromise cannot be detected by a tool that runs after boot. A bootkit controls the environment the operating system starts in and can present a stock-looking system to everything above it. The defense is measured boot with an external verifier, a ROM-anchored first verification, and robust revocation.

Firmware and component implants cannot be read by an operating-system scan. They live below the file system, and a firmware implant can present clean state to anything that asks from above. The defense is signed, measured, and attestable firmware extended down to components, and authentication of physical parts.

SIM, eSIM, and rogue-base-station attacks occur at or below the radio link, on components the operating system does not fully observe. The defense is mutual authentication of the network to the device, removal of silent downgrade to weak protocols, concealment of the long-term subscriber identifier, and hardening of SIM toolkit functionality and over-the-air provisioning. Most of it is a carrier and standards obligation.

Machine-learning model integrity cannot be assured by a signature. A poisoned, backdoored, or evaded model emits a confident wrong answer, and there is no indicator of compromise for a corrupted decision procedure. The defense is integrity and provenance attestation of model weights, training-data governance, robustness testing, and treating any model output, including a detector's own, as evidence with stated uncertainty rather than as ground truth. A detector that is itself a learned system inherits this limit.

Side-channel attacks leave no on-device artifact, because leakage is physics rather than a file. The defense is design-time and physical: constant-time implementations, microarchitectural isolation, shielding, noise injection, and physical security against remote emanation capture. Device-side software detection essentially cannot see them.

Physical-access attacks are only partially mitigated by Lockdown Mode and similar hardening. Lockdown Mode blocks configuration profiles, disables certain attachment types, and turns off 2G and 3G support (Apple, "About Lockdown Mode"). It reduces the surface; it does not eliminate it, and it does not protect against an actor who has the user's passcode.

Supply chain attacks are detectable only by integrity verification at the distribution layer, not by device-side scanning. A malicious library compiled into a legitimate application is part of the signed binary. The defense requires reproducible builds, a software bill of materials, signed provenance, and binary transparency, all upstream of the device.

Living-off-the-land attacks, the abuse of MDM profiles, accessibility APIs, and configuration profiles, are indistinguishable from their legitimate use at the artifact level. No signature can tell the difference, because the difference is in the intent of the installer. The defense is provenance and policy at the point of grant, plus user education about the risks of installing profiles and granting high-power permissions.

The common thread across these limits is that in every case the conduct occurs at a layer the device-side tool does not occupy, and the fix attaches at that layer: the carrier, the provider, the silicon, the boot ROM, the distribution pipeline, or the standard. A device-side tool is necessary

for the part of the surface it can see. It is not sufficient, and a tool that implies otherwise is claiming more than its evidence supports.

8. Why These Gaps Fall Hardest on Large Populations Under Weak Oversight

This section is analytical inference from the structural conditions described above. Where empirical claims are made, sources are cited; where the argument is structural, it is stated as such. The gaps in this paper are universal in that the trust assumptions are the same everywhere. Their consequences are not evenly distributed. Several structural conditions, present together in much of the Global South and specifically relevant to India, widen the exposure and remove the checks that limit misuse elsewhere. The argument here is about conditions, not about any particular government.

The first condition is scale. When the target population is very large, even a low base rate of targeting produces a large absolute number of affected people, and the vectors that are cheapest to operate at a distance, location tracking over signalling, identifier harvesting by rogue base stations, are precisely the ones that scale without a per-target exploit budget. The population most exposed to the carrier-side and radio-side vectors is the largest one.

The second condition is weak legal oversight. In much of the world the practical constraint on surveillance misuse is not technical but legal: a warrant requirement, judicial review, an enforced data-protection regime, an independent oversight body. Where those are weak, absent, or unenforced, the deterrent that limits abuse elsewhere is missing, and a capability that exists will be used closer to its technical limit. The device-side gap matters more here, because the legal backstop that might otherwise catch misuse after the fact is thinner.

The third condition is a fragmented and less-regulated carrier environment. The baseband and signalling vector in Section 2.3 is enabled by interconnect relationships and by Global Title leasing. A market with many operators, many interconnect partners, weaker enforcement of signalling-firewall guidelines, and looser control of leasing has a wider version of exactly the gap that Section 2.3 describes. The vector that is already invisible to the handset is also, in these conditions, less contained at the network.

The fourth condition is a device population that is older, cheaper, and less current. Where devices are kept longer, patched later or not at all, more likely to fall back to legacy radio protocols, and less likely to carry the newest hardware roots of trust, the boot chain, firmware, and radio-layer vectors in Sections 2.9, 2.11, and 2.12 are broader and the mitigations in those sections are less available. The population least able to replace its hardware is the population most exposed to the vectors that hardware change would close.

The fifth condition is the one that closes the loop. When the entity that would regulate surveillance tooling is also a customer for it, the accountability mechanism that constrains vendors elsewhere, the threat of legal or market consequence, is weakened at its source. This is a structural observation about who holds which role, not an accusation against any state. Where the roles of regulator and buyer converge, the external check on misuse is internal to the same actor, and the paper’s general point sharpens: an obligation anchored at the device cannot reach conduct in the carrier or the cloud, and where the regulator’s incentive to impose the upstream obligation is itself compromised, the upstream gap is least likely to be closed by the party best placed to close it.

The combined effect is a distributional inversion. The vectors hardest to close, the carrier-side, radio-side, and below-the-operating-system ones, are the same vectors that a device-side tool cannot observe, and they fall most heavily on large populations under weak oversight with older devices and fragmented carriers. The dominant detection model serves that population least, because it addresses the surface those users are least exposed to and misses the surface they are most exposed to. Naming the gap honestly, and naming the layer where the fix must attach, matters most exactly where the fix is least likely to arrive on its own.

9. Implications for the Field

The analysis in this paper has implications for research, regulation, and procurement.

For research, the implication is that funding should prioritize detection methods that observe structural anomalies over methods that extend signature coverage. The signature-based model has reached its structural limit. Extending coverage means adding more signatures, which does not address the class of attacks that do not reuse known infrastructure. Structural detection, observing trust assumptions, monitoring permission use, analyzing network metadata, building behavioral baselines, and, above all, moving the observation point above the attacker through attestation, is where the remaining gains are. This is a research direction, not a claim that structural detection is solved. Layers 2 through 4 are unbuilt, and their false-positive rates are unknown. The attestation-based defenses named throughout Section 2 are, in several cases, not commercially available. But the direction is where the gap is.

For regulation, the implication is that when a framework anchors accountability at the device's userspace, conduct upstream of that anchor, in the baseband, the cloud, the kernel, the boot chain, the firmware, and the radio link, is structurally unobservable. Regulators should not assume that device-side tools can cover it. The companion paper on signal provenance made the same point about the SEBI framework: accountability anchored at the broker cannot observe signal origin, because the broker never receives it. Here, accountability anchored at the device's userspace cannot observe baseband, cloud, or kernel-level attacks, because the userspace never sees them. Coverage of these vectors requires obligations at the layer where the conduct occurs: carrier-side for signalling attacks, service-provider-side for cloud compromise, platform-side and silicon-side for kernel, boot, and firmware integrity, and standards-side for the radio link. Device-side mandates are necessary but not sufficient, and a mandate that treats the device as the point of coverage will produce compliance without protection.

For procurement, the implication is that a product reporting "no findings" without stating what it examined is providing absence, not evidence. Buyers should ask what was examined and what was not. A tool that scans a backup and finds no indicators has examined the backup, not the baseband, not the cloud, not the kernel, not the boot chain. The absence of a finding is compatible with both "nothing is wrong" and "something is wrong that the tool cannot see." Procurement decisions should treat the absence of a finding as a null result, not as a clean bill of health, and should prefer a tool that states its own limits over one that offers a reassuring verdict, because the reassuring verdict is the one the method cannot support.

10. Conclusion

This paper has examined a structural limitation in signature-based spyware detection. The dominant model compares observable artifacts against a curated list of known indicators of

compromise. A match proves that a known thing was present. The absence of a match is compatible with two very different situations: nothing is wrong, or something is wrong that nobody has yet described. The delay between an attack's first use and its first published indicator is exactly the period in which signature-based detection reports nothing on an infected device. This is not a defect that better engineering removes. It is a property of the method.

The paper has argued that the correct unit of analysis is not the signature, and not the vulnerability, but the trust assumption the system was built on. Fourteen attack vectors were examined in order of sophistication, and for each the trust assumption that fails was identified, the reason signatures cannot close the gap was stated, and the shape of the defense that would close it was named, even where that defense requires hardware change, standards change, or regulatory intervention, and even where it does not yet exist. When the assumption fails, the entire class of attacks that exploit it is enabled. Signatures address the symptom; structural analysis addresses the cause.

The paper has expanded a four-layer framework for what remains observable from a device the user controls, indicator matching, permission and access observation, network metadata analysis, and behavioral baseline, and has stated for each layer what it observes, what it cannot, where its false positives come from, and why it must not claim more than its evidence supports. It has stated plainly what device-side detection cannot do, and it has shown why these gaps fall hardest on large populations under weak oversight, with fragmented carriers and older devices. The paper does not propose a solution to the hardest problems. It identifies them, names the layer where the fix must attach, and states its limits.

The companion paper on signal provenance in retail algorithmic trading identified the same structural form in a different domain. There, a regulatory framework anchored accountability at the broker, and the conduct of interest, the origin of a trading signal, occurred upstream of the broker. The framework reconstructed signal chains after the fact because nothing observed them at the time. Here, the detection framework anchors observation at the device's userspace, and the conduct of interest, the attack itself, occurs in layers the userspace cannot see. The framework reconstructs the attack after the fact because nothing observed it at the time. Both are cases in which accountability is anchored downstream of causation. The diagnostic is the same: what does the anchored entity actually receive, and is the conduct of concern present in what it receives? If the answer is no, no strengthening of obligations on that entity will close the gap. The obligation must attach at a point where the conduct is observable, or the conduct must be made observable at the point where the obligation attaches.

The paper's credibility depends on its honesty about what it does not know and cannot do. It has marked unverified claims, stated its limits, and refused to overclaim. That is the standard the field should hold itself to. A system that claims more than its evidence supports is not more secure; it is less trustworthy. The same is true of a paper.

11. References

1. 3GPP. 3G TS 33.102 V2.0.0: 3G Security; Security Architecture. April 1999. https://www.3gpp.org/ftp/tsg_sa/tsg_sa/TSGS_03/Docs/PDF/SP-99144.pdf
2. AdaptiveMobile Security (Enea). "Simjacker: Next Generation Spying Over Mobile." September 2019. https://www.enea.com/insights/simjacker-next-generation-spying-over-mobile/

3. Android Developers. “Capture and read bug reports.” https://developer.android.com/studio/debug/bug-report
4. Android Open Source Project. “Android Compatibility Definition Document.” https://source.android.com/docs/compatibility/cdd
5. Android Open Source Project. “Android Security Bulletin, December 2025.” https://source.android.com/docs/security/bulletin/2025-12-01
6. Apple. “About encrypted backups on your iPhone, iPad, or iPod touch.” Apple Support, article 108353. https://support.apple.com/en-us/108353
7. Apple. “About Lockdown Mode.” Apple Support, article 105120. https://support.apple.com/en-us/105120
8. Apple. “iCloud Data Security Overview.” Apple Support. https://support.apple.com/en-us/HT202303
9. Apple. “Profile.” Apple Developer Documentation (Device Management). https://developer.apple.com/documentation/devicemanagement/profile
10. Apple. “Protecting keys with the Secure Enclave.” Apple Developer Documentation. https://developer.apple.com/documentation/Security/protecting-keys-with-the-secure-enclave
11. Baliga, A., Ganapathy, V., and Iftode, L. “Detecting Kernel-Level Rootkits Using Data Structure Invariants.” IEEE Transactions on Dependable and Secure Computing, vol. 8, no. 5, 2011. https://doi.org/10.1109/TDSC.2010.38
12. Beer, I., and Groß, S. “A deep dive into an NSO zero-click iMessage exploit: Remote Code Execution.” Google Project Zero, December 15, 2021. https://projectzero.google/2021/12/a-deep-dive-into-nso-zero-click.html
13. CERT-EU. “Security Advisory 2021-049: Multiple Zero-Day Vulnerabilities in Apple Products.” September 14, 2021. https://cert.europa.eu/publications/security-advisories/2021-049/pdf
14. Check Point Research. “The NSO WhatsApp Vulnerability: This is How It Happened.” May 14, 2019. https://research.checkpoint.com/2019/the-nso-whatsapp-vulnerability-this-is-how-it-happened/
15. Citizen Lab. “Bad Connection: Uncovering Global Telecom Exploitation by Covert Surveillance Actors.” Report No. 192, University of Toronto, April 23, 2026. https://citizenlab.ca/research/uncovering-global-telecom-exploitation-by-covert-surveillance-actors/
16. Citizen Lab. “BLASTPASS: NSO Group iPhone Zero-Click, Zero-Day Exploit Captured in the Wild.” September 7, 2023. https://citizenlab.ca/2023/09/blastpass-nso-group-iphone-zero-click-zero-day-exploit-captured-in-the-wild/
17. Citizen Lab. “FORCEDENTRY: NSO Group iMessage Zero-Click Exploit Captured in the Wild.” September 13, 2021. https://citizenlab.ca/2021/09/forcedentry-nso-group-imessage-zero-click-exploit-captured-in-the-wild/
18. Dark Reading. “WhatsApp Secures Ban on NSO Group After 6-Year Legal Battle.” October 23, 2025. https://www.darkreading.com/cyber-risk/whatsapp-ban-nso-group-legal-battle

19. Eclypsium (Shkatov, M., and Michael, J.). “There’s a Hole in the Boot” (BootHole, CVE-2020-10713). Eclypsium, July 29, 2020. https://eclypsium.com/blog/theres-a-hole-in-the-boot/
20. ESET (Smolár, M.). “BlackLotus UEFI bootkit: Myth confirmed” (CVE-2022-21894). ESET WeLiveSecurity, March 1, 2023. https://www.welivesecurity.com/2023/03/01/blacklotus-uefi-bootkit-myth-confirmed/
21. ESET Research. “LoJax: First UEFI rootkit found in the wild, courtesy of the Sednit group.” ESET WeLiveSecurity, September 27, 2018. https://www.welivesecurity.com/2018/09/27/lojax-first-uefi-rootkit-found-wild-courtesy-sednit-group/
22. Goodfellow, I., Shlens, J., and Szegedy, C. “Explaining and Harnessing Adversarial Examples.” International Conference on Learning Representations, 2015. https://arxiv.org/abs/1412.6572
23. GSMA. “FS.11 SS7 Interconnect Security Monitoring and Firewall Guidelines.” Members only. https://www.gsma.com/solutions-and-impact/technologies/security/ [unverified]
24. GSMA. “FS.19 Diameter Interconnect Security.” Members only. https://www.gsma.com/solutions-and-impact/technologies/security/ [unverified]
25. GSMA. “FS.52 Global Title Leasing Code of Conduct.” March 2026. https://www.gsma.com/solutions-and-impact/technologies/security/gsma_resources/fs-52-global-title-leasing-code-of-conduct/
26. GSMA. “How penetration testers should approach GT Leasing.” January 2025. https://www.gsma.com/solutions-and-impact/technologies/security/ [unverified]
27. Gu, T., Dolan-Gavitt, B., and Garg, S. “BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain.” 2017. https://arxiv.org/abs/1708.06733
28. Gupta, L. “Signal Provenance in Retail Algorithmic Trading: An Architectural Gap in SEBI’s 2026 Framework and Its General Form.” SSRN Working Paper, Abstract ID 7118421, July 2026. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=7118421 (DOI 10.2139/ssrn.7118421)
29. Kaspersky Lab (GReAT). “Equation Group: The Crown Creator of Cyber-Espionage.” February 2015. https://usa.kaspersky.com/about/press-releases/equation-group-the-crown-creator-of-cyber-espionage
30. Kocher, P., Horn, J., Genkin, D., et al. “Spectre Attacks: Exploiting Speculative Execution.” IEEE Symposium on Security and Privacy, 2019. https://spectreattack.com/
31. Lipp, M., Schwarz, M., Gruss, D., et al. “Meltdown: Reading Kernel Memory from User Space.” USENIX Security Symposium, 2018. https://meltdownattack.com/
32. Markettos, A. T., Rothwell, C., Gutstein, B. F., et al. “Thunderclap: Exploring Vulnerabilities in Operating System IOMMU Protection via DMA from Untrustworthy Peripherals.” Network and Distributed System Security Symposium, 2019. https://thunderclap.io/
33. National Vulnerability Database. “CVE-2019-3568.” https://nvd.nist.gov/vuln/detail/CVE-2019-3568
34. NowSecure. “axios Was Compromised. Your Mobile App May Have Been Too.” April 2, 2026. https://www.nowsecure.com/ [unverified]

35. Ruytenberg, B. “When Lightning Strikes Thrice: Breaking Thunderbolt 3 Security.” Thunderspy, 2020. https://thunderspy.io/
36. Safeguard. “Malicious iOS SDKs and CocoaPods: A Security Report.” November 10, 2025. https://safeguard.sh/ [unverified]
37. Szegedy, C., Zaremba, W., Sutskever, I., et al. “Intriguing properties of neural networks.” International Conference on Learning Representations, 2014. https://arxiv.org/abs/1312.6199
38. van Eck, W. “Electromagnetic Radiation from Video Display Units: An Eavesdropping Risk?” Computers and Security, vol. 4, no. 4, 1985, pp. 269-286. https://www.sciencedirect.com/science/article/abs/pii/016740488590046X
39. Vassilev, A., Oprea, A., Fordyce, A., Anderson, H., Davies, X., and Hamin, M. “Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations.” NIST AI 100-2e2025, National Institute of Standards and Technology, March 2025. https://doi.org/10.6028/NIST.AI.100-2e2025
40. Yu, Y.-C., Hung, C.-Y., and Chou, L.-D. “Kernel-level hidden rootkit detection based on eBPF.” Computers and Security, vol. 157, October 2025. https://dl.acm.org/doi/10.1016/j.cose.2025.104582
41. Zaddach, J., Kurmus, A., Balzarotti, D., Blass, E.-O., Francillon, A., Goodspeed, T., Gupta, M., and Koltsidas, I. “Implementation and Implications of a Stealth Hard-Drive Backdoor.” 29th Annual Computer Security Applications Conference (ACSAC), 2013. https://dl.acm.org/doi/10.1145/2523649.2523661

12. Appendix: Unverified Claims

The following claims are marked [unverified] in the text. They rest on secondary descriptions or on documents that were not reviewed in full for this paper. They are listed here with a note on what would be required to verify each.

1. GSMA FS.11, FS.19, and FS.52 descriptions. These specifications are member-distributed and were not reviewed in full. The descriptions in Sections 2.3 and 7 rest on secondary sources. Verification requires access to the full specifications through GSMA membership or an authorized distributor.
2. The claim that GT leasing is the single most significant enabler of SS7 attacks. Quoted from a GSMA public statement; the underlying analysis was not reviewed. Verification requires the GSMA’s threat analysis.
3. IMSI catcher and cell-site simulator claims in Section 2.12. Cited as widely documented rather than to a single primary source. Verification requires primary research on cell-site simulators. (The Simjacker report is verified and cited in the references.)
4. Supply chain incidents (malicious SDKs, a compromised package) in Section 2.7. Cited from secondary security reporting. Verification requires the primary incident write-ups and the affected package advisories.
5. The description of Android bug report contents. Section headers and property names beyond the security patch level string are from public descriptions and have not been verified against a real bug report across versions and vendors.

6. The description of iPhone backup contents. The internal file layout described in Section 5 comes from published descriptions and has not been checked against real backups across iOS versions.

The following sources, marked [unverified] in an earlier draft, were verified against their primary or official sources for this version and no longer carry the mark: Spectre (IEEE S&P 2019) and Meltdown (USENIX Security 2018); Thunderclap (NDSS 2019) and Thunderspy (2020); the LoJax report (ESET, 2018); the storage-controller firmware analyses (Zaddach et al., ACSAC 2013, and Kaspersky's Equation Group analysis, 2015); BootHole (Eclypsium, 2020, CVE-2020-10713) and BlackLotus (ESET, 2023, CVE-2022-21894); the Citizen Lab "Bad Connection" report (2026); van Eck's 1985 emanation paper; Simjacker (AdaptiveMobile, now Enea, 2019); "Explaining and Harnessing Adversarial Examples" (Goodfellow, Shlens, Szegedy, 2015) and "Intriguing properties of neural networks" (Szegedy et al., 2014); BadNets (Gu, Dolan-Gavitt, Garg, 2017); and the NIST adversarial machine learning taxonomy (NIST AI 100-2e2025, March 2025).

Why the paper marks unverified claims rather than omitting them. The paper's thesis is that systems should not claim more than their evidence supports. A paper making that argument must hold itself to the same standard. Marking a claim [unverified] is not an admission of weakness; it is an act of methodological consistency. Omitting the claim would hide the uncertainty. Stating it without marking it would overclaim. Marking it states the uncertainty plainly and lets the reader judge the claim's weight. The same standard applies to the paper as to the tools it examines: the absence of verification is not evidence of falsity, and the presence of a citation is not evidence of truth. What matters is that the basis of every claim is stated, so that the reader can judge what the claim is worth.